

In-Class Lab 02 - Display Database Data in an HTML Table

Course: WEB-250

Time: About 15-20 minutes

Grading: Practice only; nothing to submit

Continue in: The same `web250/salamanders` folder

Before starting: Lab 01's connection page works.

Goal

Run SQL through PDO, retrieve individual values, and use a `foreach` loop to display them in a plain HTML table. We will build a **database summary**, directly preparing for asgn07. We are not building an animal gallery or a species-detail page.

Leave `Database.php`, the credentials, and the SQL file unchanged. Only edit `public/index.php`.

Step 1 - Retrieve one database value (about 4 minutes)

In `index.php`, locate:

```
$database = Database::connect();
```

Immediately after it, still inside the opening PHP block, add:

```
$statement = $database->query('SELECT COUNT(*) FROM salamanders');  
$salamander_count = (int) $statement->fetchColumn();
```

Replace the old success paragraph in the HTML with:

```
<p>Salamander records: <?= h((string) $salamander_count) ?></p>
```

Refresh. Expect **Salamander records: 5** with the supplied data.

You know what `COUNT(*)` does in SQL. The new parts are the PHP calls: `query()` returns a **PDOStatement object**, not the count itself. `fetchColumn()` retrieves the first column of the next result row; this query returns one row containing one value. The `(int)` cast stores that value as an integer. `query()` and `fetchColumn()`.

The expression `<?= ... ?>` means `<?php echo ...; ?>`. The supplied `h()` helper escapes text for HTML; `(string)` converts the count for its string parameter. We will look at that helper below.

Step 2 - Collect the three counts (about 4 minutes)

Replace the **opening PHP block only**, from `<?php` through its matching `?>`, with:

```
<?php
require_once __DIR__ . '/../private/initialize.php';

$database = Database::connect();

// query() returns a statement object. fetchColumn() retrieves one value.
$stmtement = $database->query('SELECT DATABASE()');
$databse_name = (string) $statement->fetchColumn();

$stmtement = $database->query('SELECT COUNT(*) FROM salamanders');
$salamander_count = (int) $statement->fetchColumn();

$stmtement = $database->query('SELECT COUNT(*) FROM habitats');
$habitat_count = (int) $statement->fetchColumn();

$stmtement = $database->query('SELECT COUNT(*) FROM
salamander_habitat_links');
$link_count = (int) $statement->fetchColumn();

// The labels are fixed, but every count came from a SQL query.
$summary = [
    ['table_name' => 'salamanders', 'row_count' => $salamander_count],
    ['table_name' => 'habitats', 'row_count' => $habitat_count],
    ['table_name' => 'salamander_habitat_links', 'row_count' =>
$link_count],
];
?>
```

Keep the HTML below it for the next step. This expands the working single-count pattern; it does not create a second connection class or switch databases.

Notice the difference between `$statement` and `$summary`: the statement is a PDO object for the most recent query. `$summary` is an ordinary PHP array that we build from the values already retrieved. It contains **three items**, one for each table we are reporting on.

We are spelling out each fixed query so the SQL-to-PHP steps stay visible. Do not generate table names from a form or URL parameter. These fixed statements do not need placeholders; queries involving visitor-supplied values will be a later prepared-statement lesson.

Step 3 - Build the table with one loop (about 6 minutes)

Keep the new PHP block. Replace everything from `<!doctype html>` to the end of `public/index.php` with:

```

<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Salamanders - Database Summary</title>
</head>
<body>
  <h1>WNC Salamanders Database Check</h1>
  <p>Database: <?= h($database_name) ?></p>

  <table>
    <caption>Database record counts</caption>
    <thead>
      <tr>
        <th scope="col">Table</th>
        <th scope="col">Rows</th>
      </tr>
    </thead>
    <tbody>
      <?php foreach ($summary as $item): ?>
        <tr>
          <th scope="row"><?= h($item['table_name']) ?></th>
          <td><?= h((string) $item['row_count']) ?></td>
        </tr>
      <?php endforeach; ?>
    </tbody>
  </table>
</body>
</html>

```

Refresh. Expect the database name `wnc_salamanders` and:

Table	Rows
salamanders	5
habitats	3
salamander_habitat_links	9

The Markdown preview above may show borders. Your actual HTML page has no CSS and may have no visible cell borders; that is correct.

Follow one iteration

At `foreach ($summary as $item):`, PHP selects one item. On the first pass, `$item['table_name']` is `salamanders`, and `$item['row_count']` contains the number retrieved from SQL. PHP outputs a `<tr>` and its cells, then repeats for the other two items.

`endforeach`; closes the loop. The colon/`endforeach` form is an alternative to braces when mixing PHP with HTML. The opening and closing table tags are **outside** the loop: one table, three body rows. The loop is processing three summary records, not the five individual salamander records. `foreach` and [alternative syntax](#).

Inspect the helper, without changing it

Open `private/functions.php`:

```
function h(string $value): string
{
    return htmlspecialchars($value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

It lets text appear as text instead of being interpreted as HTML. Use `h()` at the point where a value is output, not when it is stored in the database. It is HTML escaping, not SQL protection.

`htmlspecialchars()`.

Step 4 - Prove that the table is generated by the loop (about 2 minutes)

Temporarily swap the first and second items in `$summary`. Do not edit the HTML. Refresh: the `habitats` row now appears before `salamanders`.

Restore their order. No database records changed; only the order of the PHP display array changed. As a second observation, explain why changing a label does not change its numerical count.

Explain it to a partner (about 2 minutes)

Why do we call `fetchColumn()` after `query()`? Why does a database containing five salamander records produce only three rows in this summary table?

Ready for the next lab: All counts come from SQL, one ordinary loop builds the rows, and `index.php` still has no `try/catch`. Keep the working file for Lab 03.

Connection to asgn07

The bird project uses the same sequence: **connection -> query -> fetch a value -> build a summary array -> loop into HTML**. Its table names are `birds`, `bird_images`, and `bird_image_links`. `asgn07` also adds a purpose-label column; the loop works the same way.